

Project Antigravity

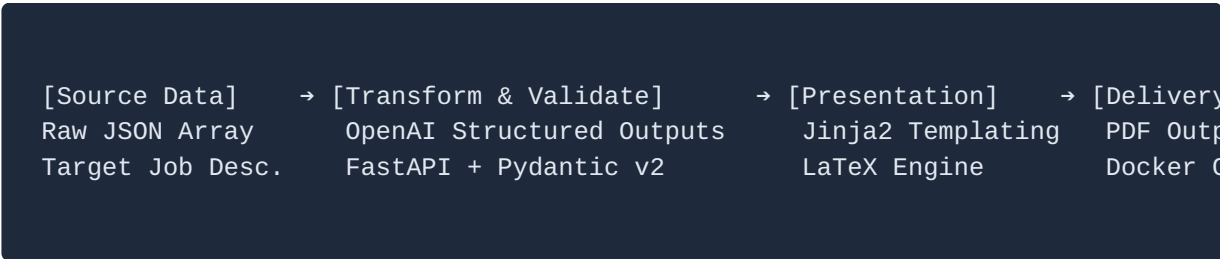
TECHNICAL ARCHITECTURE & BUSINESS BLUEPRINT: RESUME ETL PIPELINE

Executive Summary:

This document outlines the architecture for a production-grade, self-hosted data pipeline that dynamically compiles professional resumes. By treating raw career history as a structured JSON database and the presentation layer as a compiled LaTeX view, this system applies core Senior AI Data Engineering principles—using AI strictly to leverage and automate data workflows without hallucination risk. It establishes a robust, ATS-optimized, multi-mode asset generation engine.

1. System Architecture overview

The system shifts away from manual document formatting by adopting an Extract, Transform, Load (ETL) methodology. The core philosophy is the strict decoupling of data from presentation.



2. Technical Stack & Component Design

Layer	Technology	Business Purpose
Data Storage	JSON Model	Acts as the single source of truth, removing formatting constraints from raw data.
Validation (Gatekeeper)	FastAPI & Pydantic	Enforces strict schema constraints to prevent LLM hallucinations or data corruption.
AI Transformation	OpenAI (gpt-4o)	Aligns existing experience to target job descriptions deterministically using <code>response_format</code> .
Rendering	Jinja2 & LaTeX	Generates highly professional, ATS-readable documents instantly.
Infrastructure	Docker & TeX Live	Isolates compilation processes, ensuring lightweight deployment and CI/CD readiness.

3. Implementation Roadmap

Development is structured into four sequential phases to minimize integration risk and ensure deterministic behavior before adding non-deterministic AI layers.

PHASE 1 Core Engine (Deterministic Base)

- Define the Pydantic data models mirroring the LaTeX structure.
- Configure custom Jinja2 environment (overriding standard tokens to avoid LaTeX conflicts).
- Successfully compile a static `master_resume.json` into PDF locally.

PHASE 2 Infrastructure Setup

- Wrap the local compilation engine in a FastAPI REST interface.
- Create a `python:3.11-slim` Dockerfile containing minimal `texlive` dependencies.
- Implement subprocess execution for `pdflatex` with robust error handling.

PHASE 3 AI Integration (Intelligence Layer)

- Integrate OpenAI's API.
- Implement system prompts and enforce the Pydantic schema via Structured Outputs.
- Test LLM transformations to guarantee zero injection of false experiences.

- Develop routing logic to handle "Direct Custom Edit" (bypass AI) vs. "JD Tailoring" (AI transformation).
- Finalize deployment configurations.

4. Directory & Domain Structure

```
resume-etl-pipeline/
├── app/
│   ├── main.py                # FastAPI Gateway & API Routing
│   ├── models/
│   │   └── schema.py          # Pydantic Data Models (Source of Truth)
│   ├── services/
│   │   ├── llm_transformer.py # AI Transformation logic
│   │   ├── compiler.py        # Subprocess Execution for pdflatex
│   │   └── renderer.py        # Custom Jinja2 Environment
│   └── data/
│       └── master_resume.json # Raw Data Storage
├── templates/
│   └── resume_template.tex    # Core LaTeX Layout (e.g., Jake's Resume)
└── Dockerfile                 # Container Config (Python + TeX)
```

5. Strategic Value

This architecture is more than a resume builder; it is a demonstration of advanced data pipeline engineering. By abstracting the presentation layer and enforcing strict schema validation on LLM outputs, this system mitigates the primary risks of generative AI in professional applications. It stands as a scalable, automated asset ready for integration into larger autonomous multi-agent systems.